

Practical 10

1. Title

Development of REST APIs using Spring Boot and @RestController for Student and Library Management Systems

2. Aim

To develop RESTful web services using Spring Boot and the @RestController annotation for managing student records and library books, enabling CRUD (Create, Read, Update, Delete) operations through HTTP requests.

3. Learning Objectives

- To understand the concept of RESTful Web Services
- To learn how to use Spring Boot for rapid application development
- To implement REST APIs using @RestController
- To perform CRUD operations using HTTP methods (GET, POST, PUT, DELETE)
- To understand request handling using @RequestMapping, @PathVariable, and @RequestBody
- To learn JSON data exchange between client and server
- To build scalable backend services using Spring Boot

4. Software Requirements

- Framework: Spring Boot
- Language: Java (JDK 8 or above)
- Build Tool: Maven / Gradle
- IDE: Eclipse / IntelliJ IDEA / VS Code
- Server: Embedded Tomcat (comes with Spring Boot)
- Testing Tool: Postman / Browser
- OS: Windows / Linux / macOS

5. Theory Background

Spring Boot

Spring Boot is a framework that simplifies the development of Java-based applications by providing:

- Auto-configuration
- Embedded server (Tomcat)
- Minimal setup

It allows developers to build production-ready applications quickly.

REST (Representational State Transfer)

REST is an architectural style used for designing web services. It uses standard HTTP methods:

- **GET** → Retrieve data
- **POST** → Create new data
- **PUT** → Update existing data
- **DELETE** → Remove data

REST APIs usually communicate using **JSON format**.

@RestController

@RestController is a Spring annotation that combines:

- @Controller
- @ResponseBody

It returns data directly in **JSON format** instead of rendering a JSP/HTML page.

Key Annotations Used

- @GetMapping → Handle GET requests
- @PostMapping → Handle POST requests
- @PutMapping → Handle PUT requests
- @DeleteMapping → Handle DELETE requests
- @RequestBody → Read JSON data from request
- @PathVariable → Extract values from URL

6. Assignment Questions

10.1. Student Record Management REST API: Develop a REST API using Spring Boot and the @RestController annotation to manage student records in a college system. The application should

provide endpoints to add new students, retrieve student details, update student information, and delete student records. Each student record should contain attributes such as student ID, name.

Input:

Student.java

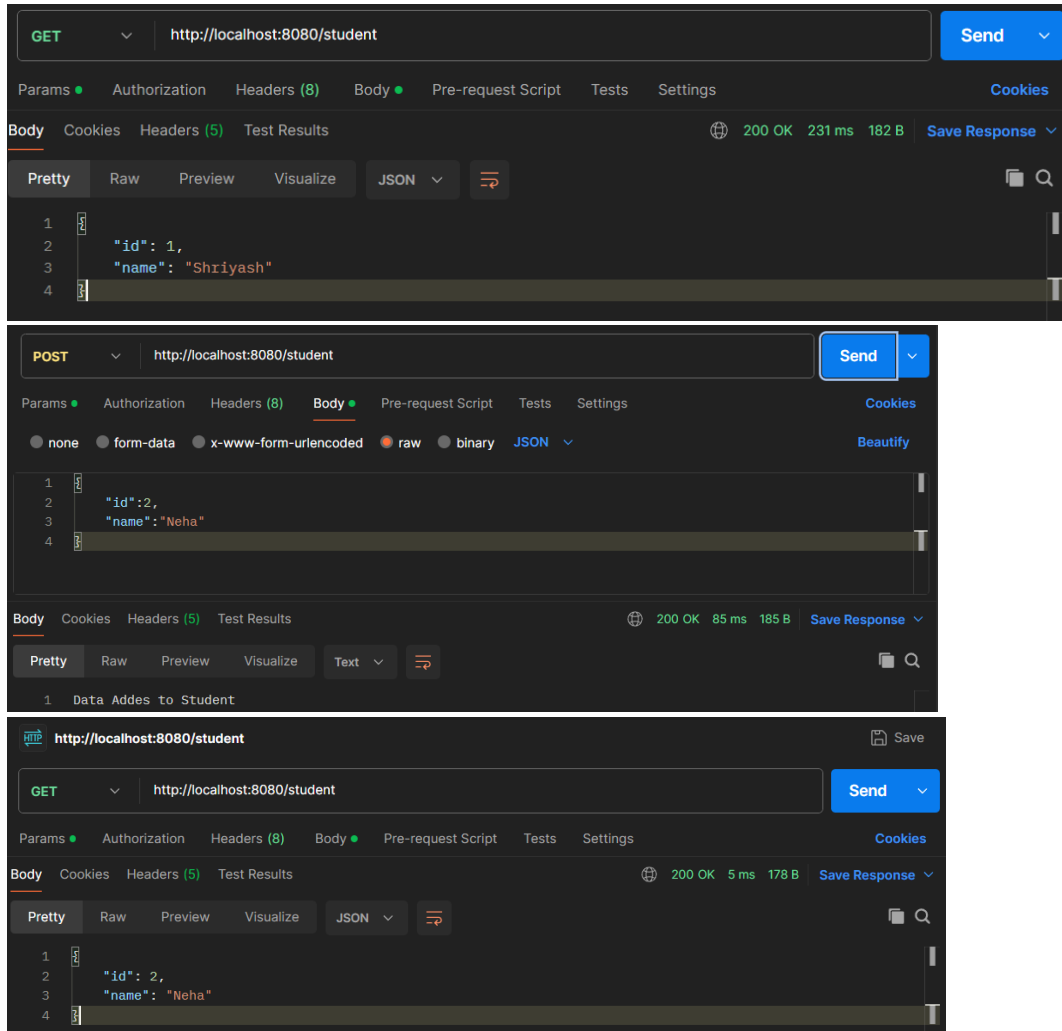
```
package com.shriyash.mca.model;
public class student {
    private int id;
    private String name;
    public student() {
    }
    public student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

StudentController.java

```
package com.shriyash.mca.controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestBody;
import com.shriyash.mca.model.student;
@RestController
@RequestMapping
public class studentController {
    student std = new student(1,"Shriyash");
    @GetMapping("/student")
    public student getStudent() {
        return std;
    }
}
```

```
}  
@PostMapping("/student")  
public String addStudent(@RequestBody student s) {  
    std = s;  
    return "Data Addes to Student";  
}  
}
```

Output:



10.2. Library Book Management REST API: Create a REST API using Spring Boot with `@RestController` to manage books in a library. The API should allow users to add new books, view the book, update book details, and remove books from the system. Each book should include fields such as book ID, title, author

Input:

Book.java

```
package com.shriyash.mca.model;

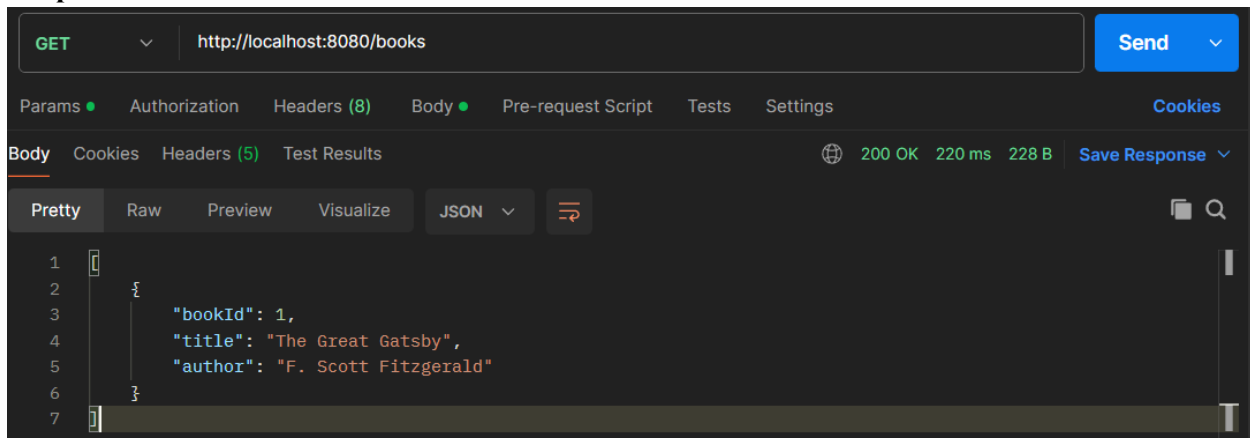
public class Book {
    private int bookId;
    private String title;
    private String author;
    public Book() {
    }
    public Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }
    public int getBookId() {
        return bookId;
    }
    public void setBookId(int bookId) {
        this.bookId = bookId;
    }
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String author) {
        this.author = author;
    }
}
```

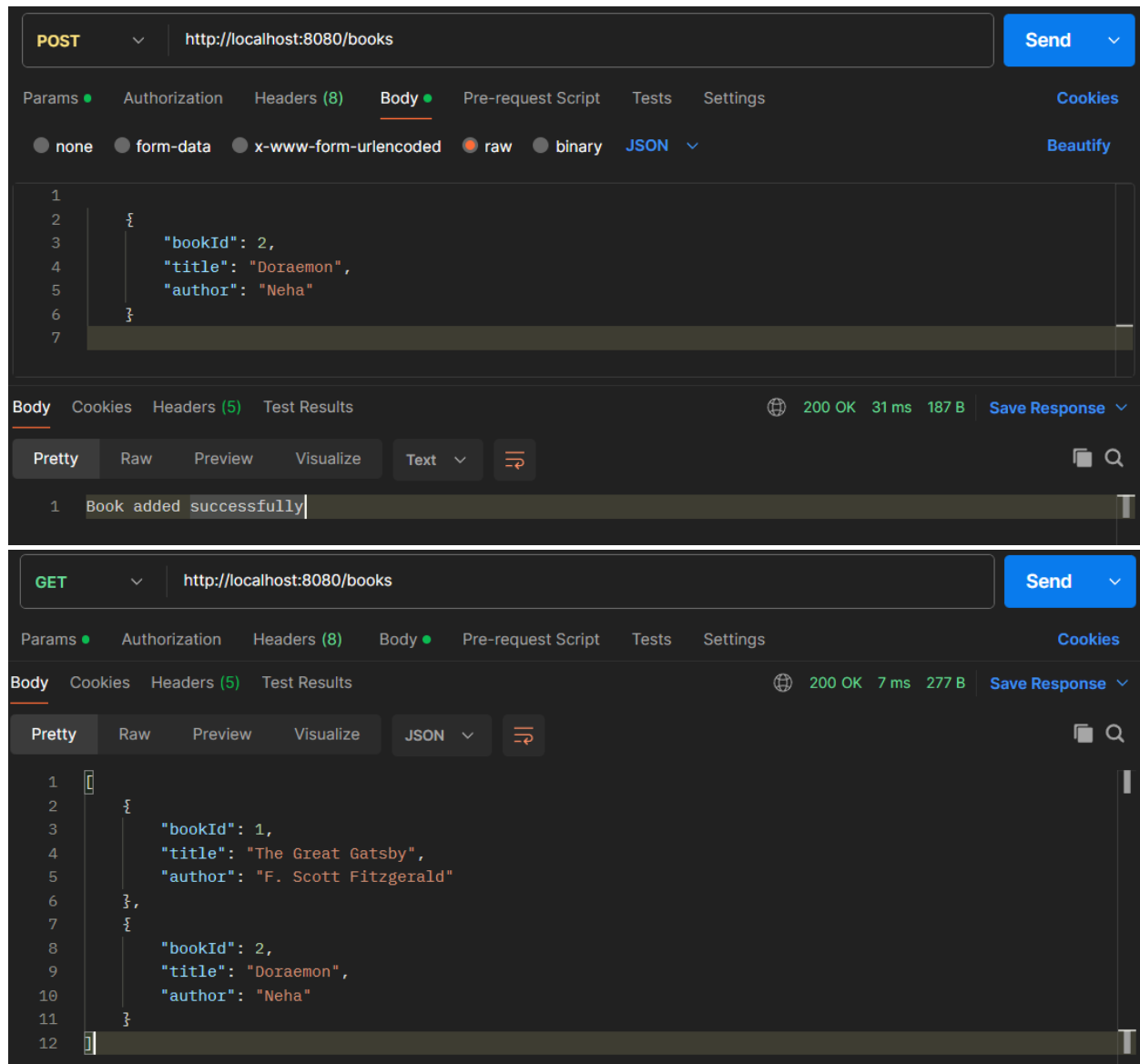
BookController.java

```
package com.shriyash.mca.controller;
import org.springframework.web.bind.annotation.*;
import com.shriyash.mca.model.Book;
import java.util.ArrayList;
```

```
import java.util.List;
@RestController
@RequestMapping("/books")
public class BookController {
    private List<Book> books = new ArrayList<>();
    public BookController() {
        books.add(new Book(1, "The Great Gatsby", "F. Scott Fitzgerald"));
    }
    @GetMapping
    public List<Book> getAllBooks() {
        return books;
    }
    @PostMapping
    public String addBook(@RequestBody Book book) {
        books.add(book);
        return "Book added successfully";
    }
}
```

Output:





Conclusion

The development of REST APIs using Spring Boot and `@RestController` successfully demonstrated how web services can be created for real-world applications such as Student Record Management and Library Book Management systems. The project provided practical knowledge of building backend applications capable of handling CRUD operations using HTTP methods like GET, POST, PUT, and DELETE. It also helped in understanding the use of important Spring Boot annotations such as `@RestController`, `@RequestMapping`, `@GetMapping`, and `@PostMapping`.